Practical 8

Aim: Implement Naïve Bayes theorem to classify the English text

Source-Code:

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    precision_score,
    recall_score
)

# Load dataset correctly by splitting the single text column on the last comma if needed
# Or reading with default comma separation if formatted properly
msglbl_data = pd.read_csv(
    'Statements_data.csv',
    header=None,
    names=['Message', 'Label']
)

# Clean up any potential parsing issues where Label might be NaN
if msglbl_data['Label'].isnull().any():
    # Fallback: manually split by the last comma if the CSV reader combined them
    raw_series = pd.read_csv('Statements_data.csv', header=None)[0]
    msglbl_data = pd.DataFrame([
        res.rsplit(',', 1) for res in raw_series
    ], columns=['Message', 'Label'])

print("Total instances in the dataset:", msglbl_data.shape[0])

# Convert labels into numbers
msglbl_data['labelnum'] = msglbl_data['Label'].str.strip().map(
    {'pos': 1, 'neg': 0}
)

# Separate input and output
X = msglbl_data['Message']
Y = msglbl_data['labelnum']

# Split the dataset
Xtrain, Xtest, Ytrain, Ytest = train_test_split(
    X, Y, test_size=0.25, random_state=42, stratify=Y
)

# Convert text into numerical features
count_vect = CountVectorizer()
Xtrain_dims = count_vect.fit_transform(Xtrain)
Xtest_dims = count_vect.transform(Xtest)

# Create and train the model
clf = MultinomialNB()
clf.fit(Xtrain_dims, Ytrain)

# Predict test data
prediction = clf.predict(Xtest_dims)
```

```
# Display evaluation metrics
print("\n******** Accuracy Metrics *********")
print("Accuracy:", accuracy_score(Ytest, prediction))
print("Recall:", recall_score(Ytest, prediction, zero_division=0))
print("Precision:", precision_score(Ytest, prediction, zero_division=0))
print("Confusion Matrix:\n", confusion_matrix(Ytest, prediction))

print("-" * 30)

# Predict a new statement
test_stmt = [input("Enter any statement to predict: ")]
test_dims = count_vect.transform(test_stmt)
pred = clf.predict(test_dims)

for stmt, lbl in zip(test_stmt, pred):
    if lbl == 1:
        print("Statement is Positive")
    else:
        print("Statement is Negative")
```

Output:

```
Total instances in the dataset: 16

******** Accuracy Metrics *********
Accuracy: 0.25
Recall: 0.5
Precision: 0.3333333333333333
Confusion Matrix:
 [[0 2]
 [1 1]]
------------------------------
Enter any statement to predict: I love this product
Statement is Positive
```